P1

Consulta 1

Item 1:

Query Query History AI Assistant

```
SET LOCAL work_mem = '64kB';

EXPLAIN (ANALYZE, buffers, verbose)
SELECT * FROM employees.employee ORDER BY hire_date;
```

Data Output Messages Notifications

Showing rows: 1 to 10

	QUERY PLAN text
1	Sort (cost=65614.50..66364.56 rows=300024 width=35) (actual time=124.158..147.491 rows=300024.00 loops=1)
2	Output: id, birth_date, first_name, last_name, gender, hire_date
3	Sort Key: employee.hire_date
4	Sort Method: external merge Disk: 13824kB
5	Buffers: shared hit=2504, temp read=6891 written=7295
6	-> Seq Scan on employees.employee (cost=0.00..5504.24 rows=300024 width=35) (actual time=0.015..11.047 rows=300024.00 loo…
7	Output: id, birth_date, first_name, last_name, gender, hire_date
8	Buffers: shared hit=2504
9	Planning Time: 0.069 ms
10	Execution Time: 157.555 ms

Query Query History AI Assistant

```
SET LOCAL work_mem = '32MB';

EXPLAIN (ANALYZE, buffers, verbose)
SELECT * FROM employees.employee ORDER BY hire_date;
```

Data Output Messages Notifications

Showing rows: 1 to 10

	QUERY PLAN text
1	Sort (cost=32798.50..33548.56 rows=300024 width=35) (actual time=59.077..107.744 rows=300024.00 loops=1)
2	Output: id, birth_date, first_name, last_name, gender, hire_date
3	Sort Key: employee.hire_date
4	Sort Method: quicksort Memory: 26115kB
5	Buffers: shared hit=2504
6	-> Seq Scan on employees.employee (cost=0.00..5504.24 rows=300024 width=35) (actual time=0.012..10.531 rows=300024.00 loo…
7	Output: id, birth_date, first_name, last_name, gender, hire_date
8	Buffers: shared hit=2504
9	Planning Time: 0.066 ms
10	Execution Time: 117.662 ms

Item 2:

Para la primer plan con un valor de work_mem = '64kb', se identificó con un nodo de ordenamiento:
Sort Method: external merge (Disk)
La cual opera bajo 2 fases, la primera en donde se cargan bloques de datos que caben en memoria, se ordenan internamente y se escriben en disco como “runs” ordenados. Para que después en la segunda fase, los runs generados se combinan progresivamente mediante un proceso de mezcla (merge), utilizando buffers limitados en memoria, hasta obtener un único archivo ordenado.

Para el segundo plan, con un valor de work_mem = '32MB', se utilizó el siguiente nodo:
Sort Method: quicksort (Memory)
Donde el ordenamiento se realiza completamente en memoria utilizando el algoritmo quicksort, evitando operaciones de escritura y lectura en disco, lo que reduce significativamente el tiempo de ejecución.

Item 3:

- Cuando la ram es insuficiente para poder hacer el ordenamiento, se usa External Merge Sort.
- Para ello, la query ejecuta en la máquina local no nos retorna el valor como tal, pero se puede estimar.
    - Tamaño de datos, tengo aprox 300024 filas con un tamaño de 35 bytes, por lo que en resumen manejo algo de 10.5MB
    - 64kB de work mem
    - Número de páginas, mi postgres maneja 8Kb de datos por página, por lo que el número de páginas en RAM es 8
    - Número de runs, esta se obtiene del total de páginas que se usarán y el número de páginas en RAM, (10.5MB/8Kb)/8, dandonos 168
    - Y para las fases es el logaritmo en base del número de páginas en ram -1 del número de runs, evidenciando la relación de runs con respecto al número de merge passes.
    - $Log_x(168) \approx 3$
- Según todo lo visto al aumentar el valor de work mem aumenta el valor de páginas en RAM, se generarían menos RUNS y por ende menos merge passes, lo que equivale a menos lecturas y escrituras temporales.

Consulta 2

Item 1:

Query Query History AI Assistant Scratch Pad

```
SET LOCAL work_mem = '64kB';

EXPLAIN (ANALYZE, buffers, verbose)
SELECT from_date, COUNT(*) FROM employees.department_employee GROUP BY from_date;
```

Data Output Messages Notifications

Showing rows: 1 to 32 Page No: 1

	QUERY PLAN text
1	Finalize GroupAggregate (cost=18151.75..19434.62 rows=6044 width=12) (actual time=121.032..138.946 rows=6393.00 loops=1)
2	Output: from_date, count(*)
3	Group Key: department_employee.from_date
4	Buffers: shared hit=2168, temp read=859 written=1829
5	-> Gather Merge (cost=18151.75..19322.80 rows=10275 width=12) (actual time=121.026..137.839 rows=12761.00 loops=1)
6	Output: from_date, (PARTIAL count(*))
7	Workers Planned: 1
8	Workers Launched: 1
9	Buffers: shared hit=2168, temp read=859 written=1829
10	-> Sort (cost=17151.74..17166.85 rows=6044 width=12) (actual time=100.461..100.932 rows=6380.50 loops=2)
11	Output: from_date, (PARTIAL count(*))
12	Sort Key: department_employee.from_date
13	Sort Method: external merge Disk: 176kB
14	Buffers: shared hit=2168, temp read=859 written=1829
15	Worker 0: actual time=81.708..82.172 rows=6377.00 loops=1
16	Sort Method: external merge Disk: 176kB
17	Buffers: shared hit=899, temp read=347 written=704
18	-> Partial HashAggregate (cost=15082.79..16667.14 rows=6044 width=12) (actual time=84.930..96.023 rows=6380.50 loops=2)
19	Output: from_date, PARTIAL count(*)
20	Group Key: department_employee.from_date
21	Planned Partitions: 4 Batches: 5 Memory Usage: 129kB Disk Usage: 5904kB
22	Buffers: shared hit=2160, temp read=815 written=1775
23	Worker 0: actual time=67.661..77.035 rows=6377.00 loops=1
24	Batches: 5 Memory Usage: 129kB Disk Usage: 3512kB
25	Buffers: shared hit=891, temp read=325 written=677
26	-> Parallel Seq Scan on employees.department_employee (cost=0.00..4110.61 rows=195061 width=4) (actual time=0.184..7.455 rows=165801.50 lo...
27	Output: employee_id, department_id, from_date, to_date
28	Buffers: shared hit=2160
29	Worker 0: actual time=0.358..6.306 rows=132370.00 loops=1
30	Buffers: shared hit=891
31	Planning Time: 0.135 ms
32	Execution Time: 140.310 ms

Query Query History AI Assistant

```
SET LOCAL work_mem = '2MB';

EXPLAIN (ANALYZE, buffers, verbose)
SELECT from_date, COUNT(*) FROM employees.department_employee GROUP BY from_date;
```

Data Output Messages Notifications

Showing rows: 1 to 10

	QUERY PLAN text
1	HashAggregate (cost=7134.05..7194.48 rows=6044 width=12) (actual time=57.968..58.835 rows=6393.00 loops=1)
2	Output: from_date, count(*)
3	Group Key: department_employee.from_date
4	Batches: 1 Memory Usage: 409kB
5	Buffers: shared hit=2160
6	-> Seq Scan on employees.department_employee (cost=0.00..5476.03 rows=331603 width=4) (actual time=0.028..14.650 rows=331603.00 loo...
7	Output: employee_id, department_id, from_date, to_date
8	Buffers: shared hit=2160
9	Planning Time: 0.172 ms
10	Execution Time: 59.496 ms

Item 2:

Para el primer plan se identificó el uso del nodo:
Partial HashAggregate
El cual trabaja en conjunto con un proceso de agregación final (Finalize GroupAggregate), evidenciando una ejecución paralela.
Adicionalmente, aparece un nodo de ordenamiento:
Sort Method: external merge (Disk)
Sin embargo, este no forma parte directa del proceso de agrupación, sino que es requerido por el operador Gather Merge, el cual necesita que los resultados parciales estén ordenados para poder combinarlos correctamente en un entorno paralelo.

Para el segundo plan se utiliza el nodo:
HashAggregate
En este caso, no se observa uso de múltiples batches ni escritura a disco, lo que indica que toda la estructura hash puede mantenerse en memoria.

Item 3:

- Cuando la memoria es limitada, PostgreSQL utiliza HashAggregate con desbordamiento a disco, lo cual corresponde al algoritmo de External Hashing.
- Cuando aparece la estrategia de Sort + GroupAggregate, esta corresponde internamente al algoritmo de External Sorting (Two-Phase Multiway Merge Sort - TPMMS). En este caso, los datos primero son ordenados (posiblemente usando external merge sort si no caben en memoria) y luego se agrupan secuencialmente.
- Cuando aparece HashAggregate con desbordamiento a disco (batches > 1), este comportamiento corresponde al algoritmo de External Hashing. En este enfoque, los datos se particionan en múltiples archivos en disco (fase de particionamiento) y posteriormente cada partición es procesada en memoria para realizar la agregación (fase de construcción y conteo).

Consulta 3

Item 1:

Query Query History AI Assistant

```
SET LOCAL work_mem = '64kB';

EXPLAIN ANALYZE
SELECT * FROM employees.employee e
JOIN employees.department_employee d ON d.employee_id = e.id;
```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No

	QUERY PLAN text
1	Hash Join (cost=11598.54..24175.04 rows=331603 width=56) (actual time=141.752..370.522 rows=331603.00 loops=1)
2	Hash Cond: (d.employee_id = e.id)
3	Buffers: shared hit=4664, temp read=3775 written=3775
4	-> Seq Scan on department_employee d (cost=0.00..5476.03 rows=331603 width=21) (actual time=0.103..50.334 rows=331603.00 loo…
5	Buffers: shared hit=2160
6	-> Hash (cost=5504.24..5504.24 rows=300024 width=35) (actual time=140.861..140.862 rows=300024.00 loops=1)
7	Buckets: 16384 Batches: 32 Memory Usage: 760kB
8	Buffers: shared hit=2504, temp written=2002
9	-> Seq Scan on employee e (cost=0.00..5504.24 rows=300024 width=35) (actual time=0.053..57.260 rows=300024.00 loops=1)
10	Buffers: shared hit=2504
11	Planning:
12	Buffers: shared hit=202 read=17 dirtied=2
13	Planning Time: 9.130 ms
14	Execution Time: 382.249 ms

```
SET LOCAL work_mem = '2MB';

EXPLAIN ANALYZE
SELECT * FROM employees.employee e
JOIN employees.department_employee d ON d.employee_id = e.id;
```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No

	QUERY PLAN text
1	Hash Join (cost=11598.54..24175.04 rows=331603 width=56) (actual time=54.920..225.899 rows=331603.00 loops=1)
2	Hash Cond: (d.employee_id = e.id)
3	Buffers: shared hit=4664, temp read=3389 written=3389
4	-> Seq Scan on department_employee d (cost=0.00..5476.03 rows=331603 width=21) (actual time=0.009..15.670 rows=331603.00 loo…
5	Buffers: shared hit=2160
6	-> Hash (cost=5504.24..5504.24 rows=300024 width=35) (actual time=54.668..54.669 rows=300024.00 loops=1)
7	Buckets: 65536 Batches: 8 Memory Usage: 3032kB
8	Buffers: shared hit=2504, temp written=1819
9	-> Seq Scan on employee e (cost=0.00..5504.24 rows=300024 width=35) (actual time=0.003..14.076 rows=300024.00 loops=1)
10	Buffers: shared hit=2504
11	Planning:
12	Buffers: shared hit=16
13	Planning Time: 0.251 ms
14	Execution Time: 238.207 ms

Item 2:

Para ambos escenarios, el nodo de join identificado es un Hash Join. En el escenario de 64 KB, se ejecuta una variante de multi-batch (o híbrida) que requiere el uso de disco, generando 16 batches debido a que los datos exceden la memoria work_mem permitida. Por el contrario, en el escenario de 2 MB, el algoritmo opera en su variante in-memory (un solo batch), lo que permite cargar la tabla completa en RAM y eliminar la necesidad de particionamiento externo, optimizando significativamente el tiempo de ejecución

Item 3:

- Con la memoria work_mem limitada a 64 KB , el algoritmo utilizado es el Hash Join. Debido a la restricción severa de RAM, el motor no puede mantener la tabla de hash completa en memoria y recurre a una variante de multi-batch que utiliza el disco para gestionar el desbordamiento de datos.
- Se generaron 16 particiones. Las particiones en PostgreSQL corresponden directamente a la Fase 1 (Particionamiento) del External Hashing. En esta etapa, el motor aplica una función hash a la clave de join (employee_id) para dividir los datos en múltiples archivos temporales en disco. Esto asegura que, en la fase posterior, cada partición individual sea lo suficientemente pequeña para cargarla en la RAM disponible (work_mem) y realizar la agregación o unión.
- En caso de que PostgreSQL hubiera optado por un Merge Join, el pre-procesamiento obligatorio habría consistido en asegurar que ambos conjuntos de datos estuvieran físicamente ordenados por sus respectivos atributos de unión, en este caso e.id y d.employee_id. Debido a que la memoria RAM es limitada (64 KB) y los atributos no cuentan con índices, PostgreSQL tendría que ejecutar un External Merge Sort previo para cada tabla. Este proceso implica una fase de generación de runs o fragmentos ordenados grabados en disco y una fase posterior de mezcla (merge).